数据结构：二叉树与搜索树
复习讲义

目录

1 树的预备知识 (Tree Preliminaries) 3
1.1 树的定义 . . . 3
1.2 术语 . . . 3
1.3 树的表示 . . . 4

2 二叉树 (Binary Trees) 4
2.1 定义与基本形态 . . . 4
2.2 二叉树性质 . . . 5
2.3 表达式树 (Expression Trees) . . . 5

3 树遍历 (Tree Traversals) 5
3.1 普通树遍历 . . . 5
3.2 二叉树中序遍历 . . . 6

4 树的应用：目录系统 7
4.1 目录列举 . . . 7
4.2 目录大小计算 . . . 8

5 线索二叉树 (Threaded Binary Trees) 8
5.1 动机 . . . 8
5.2 规则 . . . 8

6 二叉搜索树 ADT (Binary Search Trees) 9
6.1 定义 . . . 9
6.2 ADT 操作 . . . 9
6.3 查找 . . . 9
6.4 FindMin 与 FindMax . . . 10
6.5 插入 . . . 10
6.6 删除 . . . 11

6.7 惰性删除 (Lazy Deletion) 12
6.8 平均情况与最坏情况 12

7 线段树 (Segment Tree) — 二叉树的区间应用 12
7.1 动机 12
7.2 结构 12
7.3 建树 (Build) 13
7.4 区间查询 (Query) 14
7.5 单点更新 (Update) 14
7.6 扩展与注意事项 14

8 全章总结 15
8.1 复习主线 15

1 树的预备知识 (Tree Preliminaries)

1.1 树的定义

树 (Tree) 是结点的集合。该集合可以为空；若非空，则由以下部分组成：

1. 一个特殊结点 r，称为根 (root)。
2. 零个或多个非空子树 $T_1, \ldots, T_k$，每棵子树的根都由一条从 r 出发的有向边连接。

课件强调：

- 子树之间不能相连。
- 树中每个结点都是某棵子树的根。
- 含 N 个结点的树有 $N-1$ 条边。
- 通常把根画在顶部。

1.2 术语

- **结点的度 (degree of a node)**：该结点的子树个数。
- **树的度 (degree of a tree)**：所有结点度数的最大值。
- **父结点 (parent)**：拥有子树的结点。
- **孩子 (children)**：父结点各子树的根。
- **兄弟 (siblings)**：同一个父结点的孩子。
- **叶结点 (leaf / terminal node)**：度为 0 的结点。
- **路径 (path)**：从 n_1 到 n_k 的唯一结点序列 $n_1, n_2, \ldots, n_k$，满足 n_i 是 n_{i+1} 的父结点。
- **路径长度**：路径上的边数。
- **深度 (depth)**：从根到该结点的路径长度，根深度为 0。
- **高度 (height)**：从该结点到某个叶结点的最长路径长度，叶结点高度为 0。
- **树的高度/深度**：根的高度，也等于最深叶结点的深度。
- **祖先 (ancestors)**：从该结点向上到根路径上的所有结点。
- **后代 (descendants)**：该结点所有子树中的结点。

1.3 树的表示

列表表示

若根 A 有孩子 B, C, D，可写成：

$$(A(B, C, D)).$$

更深的树可继续嵌套，例如：

$$(A(B(E(K, L), F), C(G), D(H(M), I, J))).$$

缺点是每个结点需要的孩子指针数取决于分支数，结点大小不统一。

FirstChild-NextSibling 表示

每个结点只保留两个指针：

```
Element
FirstChild
NextSibling
```

- `FirstChild` 指向第一个孩子。
- `NextSibling` 指向右边的兄弟。

该表示不唯一，因为普通树中孩子顺序通常不重要。

2 二叉树 (Binary Trees)

2.1 定义与基本形态

二叉树 (Binary Tree) 是每个结点最多有两个孩子的树。通过把 FirstChild-NextSibling 表示顺时针旋转 45°，可把普通树转换成二叉树表示。

普通树与二叉树的差别

普通树中，孩子的顺序通常不重要；但在二叉树中，左孩子与右孩子不同。只有左孩子的树和只有右孩子的树是两棵不同的二叉树。

常见形态：

- **左斜树 / 右斜树 (Skewed Binary Tree)**：每个非叶结点只有一个孩子，形似链表。
- **完全二叉树 (Complete Binary Tree)**：叶结点只出现在相邻两层，且最底层结点从左到右填充。

2.2 二叉树性质

- 第 i 层最多有 2^{i-1} 个结点，其中 $i \geq 1$。
- 高度为 k 的满二叉树最多有 $2^k - 1$ 个结点，其中课件此处按层数 $k \geq 1$ 计。
- 对任意非空二叉树，若 n_0 为叶结点数，n_2 为度为 2 的结点数，则

$$n_0 = n_2 + 1.$$

证明要点：令 n_1 为度为 1 的结点数，$n = n_0 + n_1 + n_2$。边数 $B = n - 1$；另一方面所有边都从度为 1 或度为 2 的结点发出，所以 $B = n_1 + 2n_2$。两式相减得 $n_0 = n_2 + 1$。

2.3 表达式树 (Expression Trees)

表达式树也叫语法树。操作数在叶结点，操作符在内部结点。

例子：

$$A + B * C / D$$

可表示为根为 $+$，左子树为 A，右子树为除法表达式 $B * C / D$ 的树。

课件还用后缀表达式构造表达式树：

$$(a + b) * (c * (d + e)) = \texttt{a b + c d e + * *}.$$

构造规则：

1. 遇到操作数，构造单结点树并压栈。
2. 遇到二元操作符，从栈中弹出右子树和左子树，构造新树后压栈。
3. 扫描结束后，栈中唯一树即表达式树。

3 树遍历 (Tree Traversals)

3.1 普通树遍历

遍历要求每个结点恰好访问一次。

先序遍历 (Preorder)

```
void preorder( tree_ptr tree )
{
    if ( tree ) {
        visit( tree );
        for ( each child C of tree )
            preorder( C );
    }
}
```

后序遍历 (Postorder)

```
void postorder( tree_ptr tree )
{
    if ( tree ) {
        for ( each child C of tree )
            postorder( C );
        visit( tree );
    }
}
```

层序遍历 (Levelorder)

层序遍历使用队列:

```
void levelorder( tree_ptr tree )
{
    enqueue( tree );
    while ( queue is not empty ) {
        visit( T = dequeue() );
        for ( each child C of T )
            enqueue( C );
    }
}
```

对课件中编号为 1 到 7 的完全二叉树，层序遍历输出为:

$$1, 2, 3, 4, 5, 6, 7.$$

3.2 二叉树中序遍历

递归版本:

```
void inorder( tree_ptr tree )
{
    if ( tree ) {
        inorder( tree->Left );
        visit( tree->Element );
        inorder( tree->Right );
    }
}
```

迭代版本使用栈:

```
void iter_inorder( tree_ptr tree )
{
    Stack S = CreateStack( MAX_SIZE );
```

```
    for ( ; ; ) {
        for ( ; tree; tree = tree->Left )
            Push( tree, S );
        tree = Top( S ); Pop( S );
        if ( !tree ) break;
        visit( tree->Element );
        tree = tree->Right;
    }
}
```

对表达式树 $A + B * C/D$:

- 中序遍历得到: $A + B * C/D$。
- 后序遍历得到: `A B C * D / +`。
- 先序遍历得到: `+ A / * B C D`。

4 树的应用: 目录系统

4.1 目录列举

课件以 Unix 目录树为例: `/usr` 下有 `mark`、`alex`、`bill` 等目录和文件。目录列表要求深度为 d_i 的文件名前缩进 d_i 个 tab。

```
static void ListDir( DirOrFile D, int Depth )
{
    if ( D is a legitimate entry ) {
        PrintName( D, Depth );
        if ( D is a directory )
            for ( each child C of D )
                ListDir( C, Depth + 1 );
    }
}
```

时间复杂度为 $O(N)$。

课件强调: `Depth` 是内部变量，不应暴露给用户。可提供外部接口:

```
void ListDirectory( DirOrFile D )
{
    ListDir( D, 0 );
}
```

4.2 目录大小计算

若文件带大小，目录大小可通过后序递归累加：

```
static int SizeDir( DirOrFile D )
{
    int TotalSize = 0;
    if ( D is a legitimate entry ) {
        TotalSize = FileSize( D );
        if ( D is a directory )
            for ( each child C of D )
                TotalSize += SizeDir( C );
    }
    return TotalSize;
}
```

每个结点访问一次，时间复杂度为 $O(N)$。

5 线索二叉树 (Threaded Binary Trees)

5.1 动机

含 n 个结点的二叉树有 $2n$ 个孩子指针，但只有 $n-1$ 条真实边，因此有：

$$2n-(n-1)=n+1$$

个空指针。线索二叉树用这些空指针保存遍历中的前驱或后继，从而让遍历更方便。

5.2 规则

1. 若 `Tree->Left` 为空，则用它指向该结点的中序前驱。
2. 若 `Tree->Right` 为空，则用它指向该结点的中序后继。
3. 不允许 loose threads，因此线索二叉树必须有头结点；头结点的左孩子指向第一结点。

典型结点结构：

```
typedef struct ThreadedTreeNode *PtrToThreadedNode;
typedef PtrToThreadedNode ThreadedTree;
typedef struct ThreadedTreeNode {
    int LeftThread;
    ThreadedTree Left;
    ElementType Element;
    int RightThread;
    ThreadedTree Right;
};
```

其中 `LeftThread` 或 `RightThread` 为真时，对应指针是线索而不是真孩子指针。课件用 $A+B*C/D$ 的语法树展示了带头结点的线索化结果。

6 二叉搜索树 ADT (Binary Search Trees)

6.1 定义

二叉搜索树 (BST) 是一棵二叉树，可以为空；若非空，则满足：

1. 每个结点有一个整数关键字，且关键字互异。
2. 左子树中所有关键字小于根关键字。
3. 右子树中所有关键字大于根关键字。
4. 左、右子树也都是二叉搜索树。

课件给出正例与反例：例如根为 30，左子树含 $5, 2$，右子树含 40 是合法局部结构；若某结点左子树中出现大于根的关键字，则不是 BST。

6.2 ADT 操作

- `MakeEmpty(SearchTree T)`
- `Find(ElementType X, SearchTree T)`
- `FindMin(SearchTree T)`
- `FindMax(SearchTree T)`
- `Insert(ElementType X, SearchTree T)`
- `Delete(ElementType X, SearchTree T)`
- `Retrieve(Position P)`

6.3 查找

递归查找：

```
Position Find( ElementType X, SearchTree T )
{
    if ( T == NULL )
        return NULL;
    if ( X < T->Element )
        return Find( X, T->Left );
    else if ( X > T->Element )
```

```
        return Find( X, T->Right );
    else
        return T;
}
```

这是尾递归。时间和空间都与查找路径深度 d 有关：

$$T(N) = S(N) = O(d).$$

迭代版本：

```
Position Iter_Find( ElementType X, SearchTree T )
{
    while ( T ) {
        if ( X == T->Element )
            return T;
        if ( X < T->Element )
            T = T->Left;
        else
            T = T->Right;
    }
    return NULL;
}
```

6.4 FindMin 与 FindMax

- **FindMin**：不断向左走，直到左孩子为空。
- **FindMax**：不断向右走，直到右孩子为空。

二者复杂度均为 $O(d)$，其中 d 是路径深度。

6.5 插入

插入思想：先查找关键字是否存在；若不存在，查找失败时遇到的最后一个结点就是新结点的父结点。

```
SearchTree Insert( ElementType X, SearchTree T )
{
    if ( T == NULL ) {
        T = malloc( sizeof( struct TreeNode ) );
        if ( T == NULL )
            FatalError( "Out of space!!!" );
        else {
            T->Element = X;
            T->Left = T->Right = NULL;
```

```
        }
    } else if ( X < T->Element )
        T->Left = Insert( X, T->Left );
    else if ( X > T->Element )
        T->Right = Insert( X, T->Right );
    /* duplicated keys: do nothing */
    return T;
}
```

复杂度为 $O(d)$。课件特别提醒：不要忘记 `return T;`；重复关键字通常不插入。

6.6 删除

删除分三种情况：

1. **叶结点**：把父结点对应孩子指针置为空。
2. **度为 1 的结点**：用唯一孩子替代该结点。
3. **度为 2 的结点**：用左子树最大结点或右子树最小结点替代，再从子树中删除那个替代结点。

课件删除 60 的例子展示了两种合法替代：重接左子树的最大结点，或重接右子树的最小结点。

```
SearchTree Delete( ElementType X, SearchTree T )
{
    Position TmpCell;
    if ( T == NULL )
        Error( "Element not found" );
    else if ( X < T->Element )
        T->Left = Delete( X, T->Left );
    else if ( X > T->Element )
        T->Right = Delete( X, T->Right );
    else if ( T->Left && T->Right ) {
        TmpCell = FindMin( T->Right );
        T->Element = TmpCell->Element;
        T->Right = Delete( T->Element, T->Right );
    } else {
        TmpCell = T;
        if ( T->Left == NULL )
            T = T->Right;
        else if ( T->Right == NULL )
            T = T->Left;
        free( TmpCell );
```

```
        }
        return T;
    }
```

复杂度为 $O(h)$，其中 h 是树高。

6.7 惰性删除 (Lazy Deletion)

若删除操作不多，可以给每个结点增加一个标志位，表示该结点 active 或 deleted。这样删除时不实际释放空间；若同一关键字重新插入，也不用再次 `malloc`。

代价是：当 deleted 结点数量与 active 结点数量相当时，树中存在大量无效结点，会影响查找和遍历效率。

6.8 平均情况与最坏情况

BST 高度依赖插入顺序。对 $1, 2, 3, 4, 5, 6, 7$：

- 插入顺序 $4, 2, 1, 3, 6, 5, 7$ 得到较平衡的树，高度为 2。
- 插入顺序 $1, 2, 3, 4, 5, 6, 7$ 得到右斜树，高度为 6。

因此普通 BST 的最坏情况可退化成链表，操作复杂度从期望的 $O(\log N)$ 退化为 $O(N)$。

7 线段树 (Segment Tree) — 二叉树的区间应用

7.1 动机

给定数组 `A[1000000]`，需要频繁计算任意区间 $[L, R]$ 的元素和。直接扫描每次查询最坏时间为 $O(N)$，频繁查询时不可接受。

线段树要解决的问题

线段树把大区间递归拆成小区间，并在结点中保存区间聚合值。区间查询和单点更新都可以在 $O(\log N)$ 时间内完成。

7.2 结构

示例与结点含义

以数组 $A = [7, 2, 5, 8, 3]$ 为例，每个结点对应一个区间并保存该区间的和：

区间	元素	区间和
$[0,4]$	$7+2+5+8+3$	25
$[0,2]$	$7+2+5$	14
$[3,4]$	$8+3$	11
$[0,1]$	$7+2$	9
$[2,2]$	5	5
$[3,3]$	8	8
$[4,4]$	3	3
$[0,0]$	7	7
$[1,1]$	2	2

数组存储

线段树可看作完全二叉树，存储在数组 `tree[]` 中：

- 根结点 `tree[1]` 存 $[0,4]$ 的和 25。
- 对结点 `node`，左孩子为 `2*node`，右孩子为 `2*node+1`。(与完全二叉树数组存储完全一致)
- 若区间有 N 个叶子，实际结点数为 $2N-1$；工程中常分配 `4*N` 大小以简化边界。

7.3 建树 (Build)

递归建树：若 `start == end` 则为叶子，否则递归构建左右子区间后回溯合并。

```
void Build( int node, int start, int end )
{
    if ( start == end ) {
        tree[node] = A[start];
        return;
    }
    int mid = ( start + end ) / 2;
    Build( 2 * node, start, mid );
    Build( 2 * node + 1, mid + 1, end );
    tree[node] = tree[2 * node] + tree[2 * node + 1];
}
```

建树时间复杂度 $O(N)$，通常只执行一次。

7.4 区间查询 (Query)

查询区间 $[L, R]$ 时，对当前结点区间 $[\texttt{start}, \texttt{end}]$ 分三种情况：

1. **No Overlap**：完全在查询区间外，返回加法单位元 0。
2. **Total Overlap**：完全包含在查询区间内，直接返回 `tree[node]`。
3. **Partial Overlap**：部分重叠，递归查询左右孩子并合并。

```
int Query( int node, int start, int end, int L, int R )
{
    if ( R < start || end < L )     return 0;
    if ( L <= start && end <= R )   return tree[node];
    int mid = ( start + end ) / 2;
    return Query(2*node, start, mid, L, R)
         + Query(2*node+1, mid+1, end, L, R);
}
```

查询时间复杂度 $O(\log N)$。

7.5 单点更新 (Update)

将 $A[\texttt{idx}]$ 更新为 `val`：从根沿路径走到对应叶子，回溯时更新所有祖先。

```
void Update( int node, int start, int end, int idx, int val )
{
    if ( start == end ) {
        tree[node] = val;
        return;
    }
    int mid = ( start + end ) / 2;
    if ( start <= idx && idx <= mid )
        Update( 2*node, start, mid, idx, val );
    else
        Update( 2*node+1, mid+1, end, idx, val );
    tree[node] = tree[2*node] + tree[2*node+1];
}
```

单点更新时间复杂度 $O(\log N)$。

7.6 扩展与注意事项

适用范围

线段树不限于求和，适用于满足结合律的任意区间聚合操作，例如区间最小值（单位元 $+\infty$）、区间最大值（单位元 $-\infty$）、区间 gcd 等。

区间更新与 Lazy Propagation

若需对一段区间的所有元素统一修改（如区间加法），直接递归到叶子代价过高。引入**懒标记 (lazy tag)**：

- 若当前区间被完全覆盖，只更新当前结点并记录标记，不立刻下推。
- 后续访问孩子前，先将标记下推（push down）。

这样区间更新和区间查询均可保持 $O(\log N)$。

8 全章总结

8.1 复习主线

1. 普通树可空；非空树有根和若干互不相交的子树。
2. FirstChild-NextSibling 是普通树转二叉树表示的关键。
3. 遍历顺序要区分先序、后序、层序和二叉树中序。
4. 线索二叉树利用 $n+1$ 个空指针存放中序前驱/后继。
5. BST 的核心不变量是左小右大，且递归地对每棵子树成立。
6. BST 查找、插入、删除复杂度取决于树高；插入顺序可能造成退化。

“树结构的力量来自递归定义，风险也来自递归形状失衡。”